Fan Composer

NPU-Assisted Fan-Aware Soundscapes with Always-On Automatic Volume

Kyu Lee
cording.ai
October 20, 2026 | Software version 0.3.0
Preprint; not peer reviewed. Engineering feasibility report. DOI: 10.5281/zenodo.23478732

Abstract Laptop cooling fans produce persistent tonal and broadband noise. Fan Composer explores whether a quiet, fan-aware soundscape can make this noise less unpleasant without reducing its physical sound pressure. The project originated from the author's Intel Core Ultra 7 laptop having an NPU with few uses in everyday work: finding a useful local workload for that otherwise idle hardware was a primary motivation. We present a native Rust Windows application with microphone analysis, three mathematical synthesis modes, always-on automatic volume, and an OpenVINO music-policy worker that prefers an Intel NPU and falls back to CPU execution. A quiet background measurement and post-limiter stereo playback reference support conservative level tracking; uncertain estimates do not request increasing amplification. A locally generated 742-parameter neural network imitates a rule policy at 1 Hz, while FFT, gain control, limiting, and audio synthesis remain CPU DSP. Twenty-six automated tests pass. A closed-loop simulation using actual FFT analysis, synthesis, and delayed stereo leakage responds to sustained fan-level changes in all three modes. Native NPU inference and a silent physical duplex test establish execution-path feasibility. These results do not establish perceptual comfort, energy savings, reliable source separation, or long-session stability. The software, model, training script, build instructions, and validation snapshot are released for replication.

1. Motivation and scope

The starting point was practical rather than a need for a larger AI model. The author uses an Intel Core Ultra 7 laptop and noticed that its NPU was rarely engaged in ordinary work. Concern about leaving that hardware unused led to a search for a useful local application. The laptop's fan provided a continuous environmental signal and a concrete auditory problem to investigate.

The hypothesis is that adding modest, spectrally and temporally related sound may change how fan noise is perceived. The fan is not mechanically controlled, and no inverse waveform is generated to cancel pressure at the listener's ear. The application is therefore not active noise cancellation. Microphone dBFS values are not calibrated dB SPL.

Sound augmentation has prior research context. Chau et al. (2023) studied traffic-noise annoyance with added birdsong and stream sounds; that setting is not evidence about laptop fans. BNMusic (Zuo et al., 2025; revised 2026) investigates blending environmental noise into generated personalized music. Fan Composer is a smaller local engineering prototype using mathematical synthesis and a low-rate policy, without external music samples or cloud generation. No first-of-its-kind or superiority claim is made.

This report distinguishes implementation, controlled digital tests, physical stream delivery, and human perception. Only the first three receive engineering evidence here. Informal feedback on the earlier 0.1.1 sound was positive, but no controlled listener dataset has been collected.

2. System design

The Windows x64 implementation uses Rust, iced for a native UI, CPAL with shared WASAPI for device streams, and rustfft for spectral analysis. The input is downmixed to mono. A Hann STFT uses 2,048 or 4,096 samples with 75% overlap. Peak interpolation, harmonic-family evidence, temporal stability, spectral flatness, centroid, RMS, and confidence support fan-pitch estimation. A strong FFT peak alone does not identify a fan.

Audio callbacks own preallocated synthesis state and communicate through atomics and bounded SPSC queues. Model loading, model compilation, file access, UI work, and inference are outside the callbacks. The inference worker returns slowly varying parameters to the analysis worker, which publishes bounded updates to synthesis. The callback does not wait for inference.

The three modes share continuous state and safety controls. Harmonic uses a drone and overtones around an octave-adjusted fan pitch. Nature uses filtered and brown-noise textures with slow envelopes. Generative uses a reproducible motif and persistent ambient state. Phase and phrase state continue across parameter changes.

3. Always-on automatic volume

The user sets one maximum volume. There is no automatic-volume enable switch. At Start, 3.5 seconds of quiet acquisition provide a fan/background reference. The default playback gain begins 6 dB below the selected maximum, leaving room for a louder fan. Existing fixed-gain settings are migrated to a visible maximum with approximately the same starting reference gain.

Microphone and post-limiter stereo output powers are aggregated into 24 logarithmic bands from 80 to 5,000 Hz, with Hann power normalization. A four-second speaker-leakage profile is learned after playback begins; the first second allows fade and path settling. Playback is not muted during this learning period. Mean channel powers are used as a spectral reference, not as a direct recording of the sound at the microphone.

Let $P_{m,b}$ and $P_{o,b}$ denote smoothed microphone and output-reference powers in band b, and $P_{0,b}$ the quiet baseline. A conservative learned leakage coefficient gives

$$R_b = \max\bigl(P_{m,b} - k_b P_{o,b}, \varepsilon\bigr).$$

Bands dominated by predicted output or not represented by the learned profile are rejected. A weighted lower quantile of $10\log_{10}\left(\frac{R_b}{P_{0,b}}\right)$ estimates the change in background level. Coverage denotes usable baseline-band energy; it is not a probability that a source is a fan.

For a stable estimated level change ΔL, the requested gain relative to the user's maximum is

$$g^* = \mathrm{clip}(-6 + 0.5\Delta L, -30, 0) \ \ \mathrm{dB}.$$

Hysteresis and a 1.5-second stability requirement reduce rapid changes. Upward gain slew is limited to 0.45 dB/s and downward slew to 1.5 dB/s. Dominant playback can trigger a gentler 0.75 dB/s reduction to reveal the background. Missing or inconsistent references hold upward adaptation. The synthesizer adds sample-level fades and a digital peak limit; exact user mute is preserved.

This is a conservative power-leakage guard, not phase-aligned acoustic echo cancellation. Steady non-fan sounds, AGC, nonlinear speakers, device changes, or a changing acoustic path can bias the estimate. Manual remeasurement remains necessary when the background/path changes materially. Fan pitch is held between quiet measurements; level can adapt during playback.

4. NPU policy and fallback

The policy input has 16 normalized values: pitch, level, tonality, centroid, flatness, broadband energy, pitch and level rates, confidence, sine/cosine time features, three previous music-state values, mode, and preference. A fixed 16-to-32-to-6 MLP uses ReLU and sigmoid activations. Its 742 parameters occupy 2,968 bytes of float32 weights. Outputs control brightness, density, overtones, motion, reverb, and stereo width. The model does not control the maximum gain or peak cap.

Training uses 32,768 locally generated rule-teacher examples, a separate 4,096-example holdout, seed 255, and 1,200 Adam steps. Recorded holdout mean absolute error is 0.00292718, with maximum absolute error 0.05688055. These measure rule imitation, not musical quality or user comfort. There is no human-rating training set.

Rust calls the OpenVINO 2026 C API through a small dynamically loaded ABI bridge. The worker explicitly tries NPU, then OpenVINO CPU, then CPU rules if runtime/model use is impossible. Device enumeration, model compilation, a successful inference, and the runtime's execution-device property are separate checks. A request for NPU execution alone is not presented as success. Runtime failures retain synthesis continuity while changing backend.

The default policy rate is 1 Hz. Trustworthy live level estimates can update the level feature, while pitch and other contamination-sensitive features remain tied to quiet measurements. Selecting an NPU for this small workload is an exploratory use of available hardware, not evidence that the workload requires it or benefits from it. System power and fan-noise consequences remain research questions.

5. Evaluation methods

The automated suite exercises frequency accuracy, harmonic tracking, broadband rejection, synthetic fan cases, finite DSP values, peak bounds, mode continuity, parameter safety, exact mute, settings migration, and callback allocation behavior. New controller tests exercise sustained fan steps, self-output changes, short/partial-band interference, invalid references, and maximum-gain preservation.

A more integrated simulation runs the actual analyzer, stereo reference meter, gain controller, and synthesizer at 8 kHz with a 2,048-sample FFT. The microphone mixture includes stationary noise/tones and a known 70 ms delayed stereo speaker path. Each mode receives 125 seconds of virtual signal: a baseline, then a +12 dB fan step, then a −12 dB step relative to baseline. Gain is averaged over settled windows. This is not a physical acoustic test or real-time endurance run.

Native model diagnostics run 32 varying policy inputs per device and compare outputs. A named unavailable device is injected into the primary selection path to exercise real OpenVINO CPU fallback. Physical removal of the NPU or a second NPU-less PC is not tested.

The portable physical duplex test opens the laptop's actual microphone and selected Realtek speaker endpoint with maximum gain zero. It records frame counts, queue loss, stream events, callback processing budget, quiet-state observations, and real policy decisions, without saving raw microphone audio. It cannot establish audibility or acoustic feedback performance.

6. Results

Mode	Baseline	Fan +12 dB	Fan −12 dB
Harmonic	−6.00 dB	−0.14 dB	−12.19 dB
Nature	−6.00 dB	−0.01 dB	−11.98 dB
Generative	−6.00 dB	−0.09 dB	−12.15 dB

Table 1: Mean controller gain relative to the user maximum in the actual-FFT, delayed-leakage simulation. Values are gain settings, not calibrated loudness.

All modes increased and decreased gain in the intended direction while respecting the maximum. Tests of increased self-playback, with the fan unchanged, did not request increased amplification. The narrow interference tests passed; they do not establish general speech/music discrimination.

Check	Recorded result
Automated tests	26 passed; 0 failed
Release build, formatting, strict clippy	Passed
Physical silent duplex duration	45 s
Input / output frames	2165760 / 2167296
Actual NPU policy decisions	45
CPU / rule decisions during duplex	0 / 0
Input / output-reference queue drops	0 / 0
Stream fault / callback over-budget	0 / 0
Maximum observed callback processing	619 microseconds
Reported startup input Xrun events	1
Quiet observations after 4 seconds	0

Table 2: Bounded engineering observations for the release candidate. The physical test was silent and is not a listening result.

The same-input native policy comparison produced a maximum CPU/NPU absolute difference of 0.00055206 across 32 inputs. Both actual-device inference and injected-unavailability CPU fallback completed. These numerical checks do not demonstrate a preferred musical result.

A historical 0.2.0 physical test also completed 45 NPU policy decisions in 45 seconds of silent duplex I/O with one startup Xrun. The current release retains explicit stream-event reporting. An observed callback processing maximum is not a latency distribution, a total end-to-end latency, or a guarantee against future glitches.

7. Limits of the evidence

No controlled comfort study, calibrated sound-pressure measurement, CPU-load comparison, system/NPU power comparison, or one-hour/eight-hour endurance session is reported. A quiet microphone measurement can fail to identify a reliable fan pitch; the application does not invent one. Physical fan/noise level steps and acoustic self-output rejection need real-room testing.

The earlier informal user report described audible, generally acceptable sound in all three modes. That feedback predates automatic volume and must not be treated as validation of version 0.3.0, a clinical outcome, or a participant study.

8. Interface and reproducibility

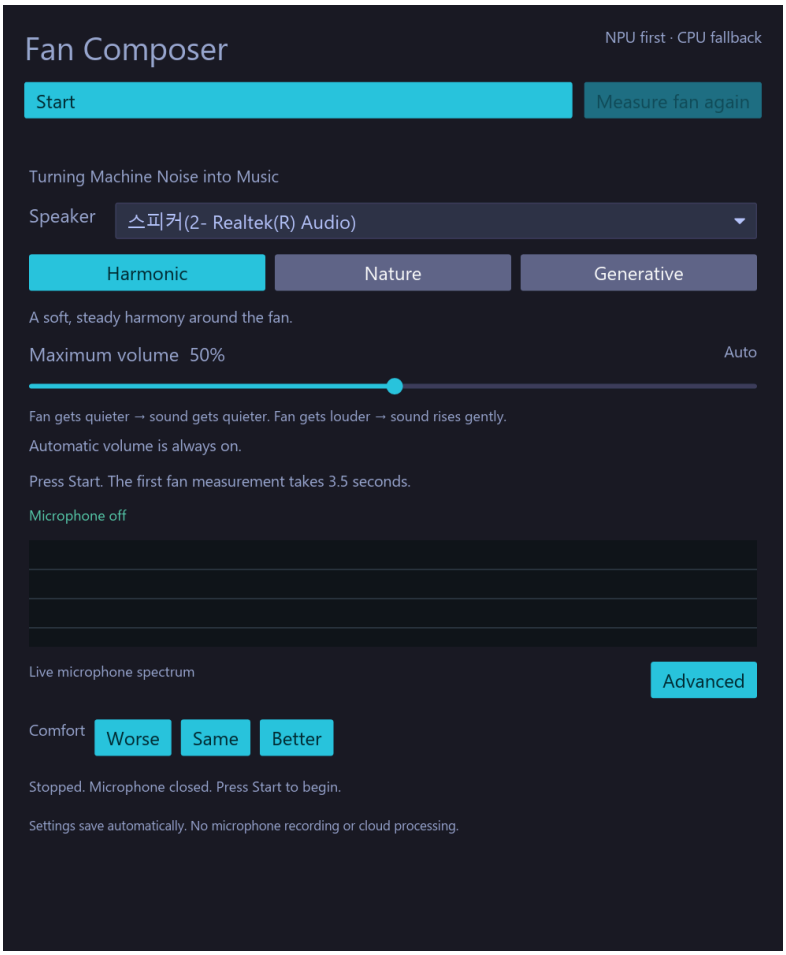


Figure 1: Actual native Windows UI capture in the stopped state. Start/Stop remains visible, automatic volume is always enabled, the user sets a maximum, and technical controls are collapsed under Advanced. Machine-specific speaker names are visible.

The primary screen exposes three modes, one maximum-volume control, speaker selection, Start/Stop, remeasurement, a compact spectrum, and an actual-backend status. Detailed inference counts, stream diagnostics, microphone selection, and experimental controls remain available without occupying the primary screen.

The source repository is github.com/zendoclab/fan-composer. It includes Rust source, a locked dependency graph, the model, offline teacher training, tests, English build/use guides, and a sanitized validation report. Local device inventories, raw endpoint IDs, user settings, credentials, and build directories are excluded.

On Windows with Rust MSVC and Visual Studio C++ tools:

```
cargo build --release --locked
cargo test --release --locked
cargo clippy --all-targets --locked -- -D warnings
```

The portable release includes the EXE, model, OpenVINO CPU/NPU runtime, and third-party licenses. Python is an offline build/training dependency, not an application runtime requirement. Launching the app does not start capture/playback. Raw microphone audio is not saved or sent to a cloud service.

9. Next experiments

The main perceptual hypothesis remains open: with fan state and total listening level controlled, does Fourier-adaptive or NPU-policy-guided sound reduce annoyance compared with fan noise alone and a fixed ambient track?

A planned within-listener experiment compares A: fan only, B: fan plus fixed ambient, C: fan plus a CPU rule-guided adaptive soundscape, and D: fan plus NPU-model-guided soundscape. Presentation order should be randomized and fan conditions matched. Ratings from 1 to 7 should cover annoyance, distraction, naturalness, harmony, loudness burden, and willingness to use the app for a long session. This protocol is proposed, not executed.

Rule imitation makes a large C-versus-D musical difference unlikely unless the policy or learned objective changes. It is therefore important to compare execution efficiency separately from listener preference, and to avoid attributing a louder or simply different soundtrack to NPU quality. Future models could use carefully collected preference data, with conservative validation to avoid overfitting.

An energy experiment should compare application closed, analysis/synthesis only, CPU policy, and NPU policy under comparable external load and fan state. Wall power or trustworthy whole-system telemetry is preferable to fabricated device-power estimates. No NPU Turbo mode is deliberately requested. Additional work should test device loss, reference drop recovery, microphone processing, room/path changes, sleep/resume, and long sessions.

10. Conclusion

Fan Composer supplies a working native research prototype for converting laptop-fan characteristics into bounded, continuous soundscapes. Its always-on volume controller is verified under controlled closed-loop simulations, and its NPU-first policy is backed by actual device execution rather than a requested-device label. The project demonstrates an implementable local use of otherwise underused NPU hardware. Whether this improves comfort or overall efficiency remains to be measured.

11. Availability and disclosures

Author: Kyu Lee, cording.ai. Implementation and documentation were developed with assistance from OpenAI Codex. The reported tests are local engineering measurements and have not been independently replicated. No controlled human-subject study is reported.

Source and locally generated model: MIT license. This preprint: CC BY 4.0. Bundled runtime components retain their respective licenses. No commercial or institutional endorsement by Intel, Microsoft, or the cited tool maintainers is implied.

12. References

1. Zuo, C., Møller, M. B., Martínez-Nuevo, P., Huang, H., Wu, Y., and Zhu, Y. (2025; revised 2026). **BNMusic: Blending Environmental Noises into Personalized Music**. doi:10.48550/arXiv.2506.10754.
2. Chau, C. K., Leung, T. M., Chung, W. K., and Tang, S. K. (2023). **Effect of perceived dominance and pleasantness on the total noise annoyance responses evoked by augmenting road traffic noise with birdsong/stream sound**. Applied Acoustics, 213, 109650. doi:10.1016/j.apacoust.2023.109650.
3. OpenVINO (2026). **Core C API** and **NPU device documentation**. docs.openvino.ai/2026. Accessed October 20, 2026.
4. CPAL contributors. **CPAL 0.18.2 API documentation**. docs.rs/cpal/0.18.2.
5. rustfft contributors. **rustfft 6.4.1 API documentation**. docs.rs/rustfft/6.4.1.
6. iced contributors. **iced 0.14 API documentation**. docs.rs/iced/0.14.0.